

快速入门

文件管理 ▾

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

语法文件开发

MT 语法文件 (.mtsx) 用于文本编辑器的语法高亮功能，其核心是通过正则表达式按一定规则进行匹配，并将匹配到的文本进行着色处理，因此你需要熟练掌握正则表达式才能进行编写。

目录

- 变更记录
- ★ 正则表达式
 - 修饰符
 - keywordsToRegex 函数
 - include 函数
- ★ 匹配器
 - 1. match 匹配器
 - 2. start-end 匹配器
 - 3. group 匹配器
 - 4. number 匹配器
 - 5. builtin 匹配器
 - 6. include 匹配器
- ★ 核心逻辑
 - 匹配流程
 - 优先级规则
 - 示例分析
 - 总结
- ★ 文件结构一览
- 属性：name
- 属性：hide
- 属性：ignoreCase
- 属性：styles
 - 颜色格式
 - 行背景色
 - 风格的向下传播
- 属性：comment
- 属性：bracketPairs
- 属性：lineBackground

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

- 用于正则表达式
- 用于匹配器
- 用于 childrenSyntax 映射
- 属性：codeFormatter
- 属性：codeShrinker
- 指定最低 MT 版本
- 预处理指令
 - 支持的指令
 - 条件表达式
 - 用法示例
 - 注意事项
 - 与 require 指令的区别
- 安装语法高亮文件
- 附录
 - 内置的风格
 - 内置的匹配器
 - LSP 插件

变更记录

v2.26.9 版本

- start-end 匹配器新增 childrenSyntax 用法与 missingStyle 属性，支持在匹配范围内使用其它语法文件进行高亮
- defines 支持定义可供 childrenSyntax 复用的动态映射集合
- match 匹配器的 parseColor 功能新增 RGBf、ARGBf 和 RGBAf 类型，ARGB 不再等同于 XRGB

v2.19.4 版本

- 语法文件新增预处理指令功能，支持条件编译

v2.19.0 版本

- number 匹配器新增单引号分隔符选项

v2.18.5 版本

- match 匹配器的 parseColor 功能新增 RGBX 和 XRGB 类型

v2.18.4 版本

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

v2.15.3 版本

- match 匹配器的 [parseColor 功能](#)会自动附带颜色选择器功能

v2.16.5 版本

- 支持[指定安装该语法文件的最低 MT 版本](#)

v2.16.0 版本

- 主属性 colors 变更为 styles，各个功能的属性 color 变更为 style
- [styles](#) 中除了可以定义字体颜色，还支持设置粗体、斜体、下划线、删除线、背景颜色、行背景
- 新增 [bracketPairs 属性](#)用于指定自动高亮括号对
- 新增 [group 匹配器](#)，可将多个匹配器按指定规则进行处理

- match 匹配器支持定义[捕获组子匹配器](#)，新增 [parseColor 功能](#)
- start-end 匹配器新增 [mustMatchEnd 属性](#)、[childrenStyle 属性](#)
- start-end 匹配器的 contains 块新增 [EndMatcher 标记](#)和 [Fail 标记](#) 功能

为了兼容之前的语法文件，你仍然可以使用 colors 和 color 来表示 styles 和 style，但还是建议修改为最新的语法，MT 文本编辑器已对这两个属性添加了删除线来表示不推荐使用。

v2.15.4 版本

- start-end 匹配器新增 [endPriority 属性](#)

v2.14.3 版本

- 新增 [defines 功能](#)，支持匹配器命名与递归
- 新增 [include 匹配器](#)与 [include 函数](#)，与 defines 功能配合使用
- match 匹配器新增 [recordAllGroups 属性](#)
- start-end 匹配器新增 [matchEndFirst 属性](#)

v2.13.2 版本

- 新增 [number 匹配器](#)，用于快速匹配编程语言数字

快速入门

文件管理 ▾

ROOT 相关

MT2 文件夹

书签手势说明

文件打开方式

压缩文件相关

批量重命名

文本编辑器

正则表达式

语法文件开发

Hex 编辑器

文件对比器

终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

★ 正则表达式

首先介绍下 MT 语法文件的正则表达式写法，主要有两种：

```
"正则表达式\\s+"  
/正则表达式\s+/
```

第一种是把正则表达式写在双引号之间，与 Java 代码中字符串的写法完全一样，缺点是需要对 `\` 和 `"` 等字符进行转义，因为在正则表达式中经常会用到 `\`，转义会影响可读性，正常我们推荐第二种写法。

第二种是把正则表达式写在 `/` 之间，如果正则表达式中包含 `/`，你需要把它写成 `\/`，其它字符均无需转义。

```
"abc/def\\s\""  
/abc\/def\s"/
```

你可以将多个子表达式使用 `+` 连接，子表达式可以写在不同行，以下三个正则表达式是一样的：

```
/123456789/
```

```
"123" + /456789/
```

```
/132/  
+ /456/  
+ /789/
```

修饰符

你可以在正则表达式中使用以下修饰符：

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

符	
(?i)	忽略大小写。例如，"(?i)hello" 将匹配 "hello", "Hello", "HELLO" 等
(?m)	多行模式。在这种模式下， <code>^</code> 和 <code>\$</code> 将匹配每一行的开始和结束，而不仅仅是整个输入的开头和结尾
(?s)	DotAll 模式。在这种模式下， <code>.</code> 将匹配任何字符，包括换行符
(?u)	启用后不区分大小写的匹配将以符合 Unicode 标准的方式完成

keywordsToRegex 函数

对于大多数语言，我们需要对该语言的关键字进行高亮，因此需要编写正则表达式对关键字进行匹配。例如一个语言包含关键字 `if` 和 `else`，那么我们编写的正则表达式应该是 `/\b(if|else)\b/`。

但将关键字换成 `move` 和 `move-wide`，如果你编写的正则表达式是 `/\b(move|move-wide)\b/`，那么你会发现这个正则表达式只能匹配到 `move`，永远无法匹配到 `move-wide`。

为了正确匹配，你需要将它写成 `/\b(move-wide|move)\b/`，但这样性能不是很好，更高效率的写法是 `/\b(move(-wide)?)\b/`。

再进一步优化下，写成 `/(?:\b(?:move(?:-wide)?)\b)/`，其中最外层的括号是为了方便其作为一个子表达式连接其它子表达式。

这还只是仅有两个关键字的情况，如果有很多个关键字，那么情况会更加复杂，为了解决该问题，你可以使用 `keywordsToRegex` 函数，它自动帮你完成以上各个优化工作，并返回一个正则表达式。

```
keywordsToRegex(  
    "while volatile void try true transient throws throws  
    "synchronized switch super strictfp static short  
    "public protected private package null new native  
    "interface int instanceof import implements if go  
    "float finally final false extends enum else doub  
    "default continue const class char catch case byte  
    "boolean assert abstract"  
)
```

快速入门

文件管理 ▾

ROOT 相关

MT2 文件夹

书签手势说明

文件打开方式

压缩文件相关

批量重命名

文本编辑器

正则表达式

语法文件开发

Hex 编辑器

文件对比器

终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

因为 `keywordsToRegex` 函数返回的是一个正则表达式，所以你可以把它当作一个子表达式去连接其它表达式，十分方便。

```
/(?m)^[ \t]*/ + keywordsToRegex("if else")
```

include 函数

你可以把多次用到的正则表达式片段写在 `defines: []` 块中：

```
defines: [  
  "name": /正则表达式abc/  
]
```

然后通过 `include` 来引用这个片段

```
include("name") // 相当于上面的 /正则表达式abc/  
  
/(?m)^[ \t]*/ + include("name") // 一样可以和其它正则表达式
```

关于 `defines` 和 `include` 在[本文后面](#)有更详细的介绍。

★ 匹配器

正则表达式指定了用什么样的规则去查找文本，而匹配器则是指定了在找到文本之后，用什么样的规则去给文本上色，因此匹配器是整个语法文件的核心。

当前 MT 的语法引擎支持六种类型的匹配器：

- `match` 匹配器
- `start-end` 匹配器
- `group` 匹配器
- `number` 匹配器
- `builtin` 匹配器
- `include` 匹配器

1. match 匹配器

快速入门

文件管理 ▾

ROOT 相关

MT2 文件夹

书签手势说明

文件打开方式

压缩文件相关

批量重命名

文本编辑器

正则表达式

语法文件开发

Hex 编辑器

文件对比器

终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

match 匹配器使用指定的正则表达式进行匹配，一旦匹配成功，会根据匹配组的风格对文本进行着色，如果要对整个匹配到的文本进行着色，则对应的捕获组序号为 0，具体请参考以下例子：

```
{match: /[a-z]+\d+[a-z]+/, 0: "string"}
```

- 高亮结果：abc123456def

```
{match: /[a-z]+\d+([a-z]+)/, 1: "string"}
```

- 高亮结果：abc123456def

```
{match: /([a-z]+)(\d+)([a-z]+)/, 1: "string", 2: "error"}
```

- 高亮结果：abc123456def

如果同时使用捕获组序列号 0 与其它序列号，则序列号 0 对应的风格相当于整个匹配文本段的默认风格。

```
{match: /([a-z]+)(\d+)([a-z]+)/, 0: "string", 2: "error"}
```

- 高亮结果：abc123456def（abc与def使用默认风格进行着色）

recordAllGroups 属性

可选属性，默认值为 false，若填写则必须在 match 属性后面！

正则表达式的每个子匹配组只能记录最后一个位置，如果你希望记录子匹配组的所有匹配位置并进行着色，将 recordAllGroups 属性设为 true 即可。

例如你想用正则表达式 a(1|2)+b 去匹配文本，并让其中的 1 显示为黄色，2 显示为红色，你可能会这么写 match 匹配器：

```
{match: /a(?: (1)|(2))+b/, 1: "meta", 2: "error"}
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

次 1 和 2，但只记录了最后一次。

添加 `recordAllGroups` 属性即可解决：

```
{match: /a(?: (1)| (2))+b/, recordAllGroups: true, 1: "number", 2: "string"}
```

- 高亮结果：a121212b

捕获组子匹配器

你可以定义捕获组子匹配器，来对捕获组范围内的文本进行二次匹配与上色。

例如你需要匹配一段使用 `< >` 包裹起来的文本，然后对尖括号中间的数字和字母使用不同颜色进行上色。

如果使用单一正则表达式来实现会非常复杂，而使用捕获组子匹配器则很容易实现：

```
{
  match: /<(.*?)>/
  1: {
    match: /(\d+)|([a-z]+)/
    1: "number"
    2: "string"
  }
}
```

同个捕获组可定义多个子匹配器，因此上面的例子可以分开写：

```
{
  match: /<(.*?)>/
  1: {match: /\d+/, 0: "number"}
  1: {match: /[a-z]+/, 0: "string"}
}
```

- 高亮结果：<啊abc123456哦def>

同个捕获组也可以同时定义风格和子匹配器，此时的风格表示该匹配组的默认风格：

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

```
1: "error"
1: {match: /\d+/, 0: "number"}
1: {match: /[a-z]+/, 0: "string"}
}
```

- 高亮结果：<啊abc123456哦def>

parseColor 功能

该功能用于根据代码内容动态设置文本前景色和背景色，我们直接使用例子来说明：

如果你对 HTML 有所了解，那么应该知道 #0000FF 是一个表示蓝色的代码，我们可以使用 match 匹配器配合 parseColor 功能来显示此类颜色：

我们这边只考虑 6 位长度的颜色，使用正则表达式 /(#[0-9A-Fa-f]{6})\b/ 来匹配：

- 捕获组0: 匹配整个文本
- 捕获组1: 匹配 #
- 捕获组2: 匹配 0000FF

将解析到的颜色作为前景色高亮整个文本

```
{match: /(#[0-9A-Fa-f]{6})\b/, 0: "parseColor(2,_,HEX)"}
```

- 高亮结果：#0000FF

将解析到的颜色作为前景色仅高亮 # 号

```
{match: /(#[0-9A-Fa-f]{6})\b/, 1: "parseColor(2,_,HEX)"}
```

- 高亮结果：#0000FF

将解析到的颜色作为背景色高亮整个文本

```
{match: /(#[0-9A-Fa-f]{6})\b/, 0: "parseColor(_,2,HEX)"}
```

- 高亮结果：#0000FF（文字显示不清）

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

```
{match: /(#[0-9A-Fa-f]{6})\b/, 0: "parseColor(auto,2
```

- 高亮结果：**#0000FF**（这下显示清晰了）

parseColor 共有 4 个参数，分别是：

- ①前景色和②背景色
 - 可以是： 匹配组序号 、 _ 、 auto
- ③颜色数值格式
 - 可以是：HEX、HEXA、RGB、HSL、HSV、RGBA、HSLA、HSVA、RGBX、XRGB、ARGB、RGBf、ARGBf、RGBAf
- ④基础风格
 - 填写风格名称

对于①前景色和②背景色：

- 如果填写 匹配组序号 ，则表示使用该匹配组的文本解析并获取颜色，使用什么格式去解析由③颜色数值格式来决定；
- 如果填写 _ ，则表示使用④基础风格的对应颜色；
- 如果填写 auto ，则会根据另一颜色生成一个适合显示的颜色，例如前景色是白色，就会生成黑色的背景色，确保你能看到白色文字，反之亦然；
- 必须至少一个是 匹配组序号 ，不能只有 _ 和 auto ，这样没有意义！

对于③颜色数值格式：

颜色数值格式决定如何从捕获组文本中依次解析颜色分量。除十六进制格式外，下表使用三种取值范围：

- 整数值： 0 ~ 255 ；
- 归一化值： 0 ~ 1 或 0% ~ 100% ，例如 0.5 与 50% 等价；不同分量可以混合使用两种写法；
- 色相值： 0 ~ 360 ，其中 360 与 0 等价。

格式	分量顺序与取值范围	示例
HEX	3、6 位为 RGB； 4、8 位为 ARGB	F00 、 FF0000 、 80FF0000
HEXA	3、6 位为 RGB； 4、8 位为 RGBA	F00 、 FF0000 、 FF000080

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

RGB	R、G、B 均为整数值	255,0,0
RGBA	R、G、B 为整数值，A 为归一化值	255,0,0,50%
ARGB	A 为归一化值，R、G、B 为整数值	50%,255,0,0
RGBX	R、G、B、A 均为整数值	255,0,0,128
XRGB	A、R、G、B 均为整数值	128,255,0,0
RGBf	R、G、B 均为归一化值	1,0,0
RGBAf	R、G、B、A 均为归一化值	1,0,0,50%
ARGBf	A、R、G、B 均为归一化值	50%,1,0,0
HSL	H 为色相值，S、L 为归一化值	0,100%,50%
HSLA	在 HSL 末尾增加归一化的 A	0,100%,50%,50%
HSV	H 为色相值，S、V 为归一化值	0,100%,100%
HSVA	在 HSV 末尾增加归一化的 A	0,100%,100%,50%

在 RGB 系列中：

- `A` 表示归一化的透明度；
- `X` 表示使用 `0~255` 整数值的透明度；
- 小写 `f` 表示 R、G、B 改用归一化值。

除了 HEX 和 HEXA，其余格式都会从捕获组文本中依次提取所需的数值，分隔符不限于逗号。例如 RGB 格式中的 `255,0,0`、`255 0 0` 和 `255|0|0` 均可解析。

版本兼容性：

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

- ARGB 的当前语义以及 RGBf、ARGBf、RGBAf 需要 MT 2.26.9 或更高版本。使用这些格式时，应在文件开头声明对应的 `// require MT >= ...`。

对于④基础风格：

- 如果颜色数值解析失败，被高亮的文本将会直接使用默认风格；
- 被高亮的文本会继承基础风格的加粗、斜体、下划线、删除线属性；
- 如果前景色或背景色参数填写的是 `_`，那么被高亮的文本将会使用基础风格的前景色或背景色。

从 2.16.6 版本开始该功能还附带提供颜色选择器功能（选中颜色数值部分的文本即可在弹出菜单中看到）

2. start-end 匹配器

start-end 匹配器用于处理具有明确起点和终点的文本范围。起点与终点之间的正文既可以交给 `contains` 中的子匹配器处理，也可以通过 `childrenSyntax` 复用另一个语法文件；这两种用法互斥，不能同时使用。

通用属性

```
{
  start: 起点匹配器
  end: 终点匹配器
  style: "风格名称, 可空"
  childrenStyle: "风格名称, 可空"
  mustMatchEnd: false // 是否必须匹配到终点，默认为false
}
```

- `start` 和 `end` 分别定义范围的起点与终点
- `style` 用于指定 `start`、`end` 和正文的默认风格
- `childrenStyle` 只指定正文的基础风格，优先级高于 `style`
- `mustMatchEnd` 决定终点匹配失败时，是否让整个 start-end 匹配失败

默认情况下，到达文本末尾也会被认为是匹配到终点。例如 Java 块注释只有 `/*` 而没有 `*/` 时，从 `/*` 开始到文本末尾仍会被

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

运行行为支持自定义正则语法，也能做正则匹配。想不开正则表达式的话，时存在起点和终点才能成立，可以设置 `mustMatchEnd: true`。

contains 用法

```
{
  start: 起点匹配器
  end: 终点匹配器
  style: "风格名称, 可空"
  childrenStyle: "风格名称, 可空"
  matchEndFirst: false // 是否优先确定终点, 默认为false
  endPriority: 0 // 终点匹配器优先级, 默认为0
  contains: [
    子匹配器1
    子匹配器2
    .....
  ]
}
```

使用 `contains` 时，默认算法是首先匹配起点，然后让终点匹配器与子匹配器不断竞争。子匹配器可以匹配多次，一旦终点匹配成功或到达文本末尾，整个范围结束。

通俗点说就是先匹配起点，然后终点匹配器和子匹配器一轮又一轮相互竞争，谁匹配位置更靠前就用谁，直到终点匹配器成功匹配或者到达文本末尾。

以代码字符串为例，字符串开头是一个 `"`，结尾也是一个 `"`，那么就可以这么写：

```
{
  start: {match: /\"/}
  end: {match: /\"/}
  style: "string"
}
```

高亮结果如下：

- "abcdefg"
- "abcd\"efg"

可以看到第二个字符串没有识别出转义符，导致高亮有误，此时我们就需要子匹配器去和终点匹配器竞争。

快速入门

文件管理 ▾

[ROOT 相关](#)[MT2 文件夹](#)[书签手势说明](#)[文件打开方式](#)[压缩文件相关](#)[批量重命名](#)[文本编辑器](#)[正则表达式](#)[语法文件开发](#)[Hex 编辑器](#)[文件对比器](#)[终端模拟器](#)

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

```
end: {match: /\\"/}  
style: "string"  
contains: [  
    {match: /\\".\\/}  
]  
}
```

这样就可以正确匹配了，我们来分析一下：

```
" a b c d \ " e f g "  
0 1 2 3 4 5 6 7 8 9 10  >> 字符位置参照
```

1. 起点匹配器搜索到位置为 0 的 " ；
2. 子匹配器和终点匹配器从位置 1 开始搜索；
3. 子匹配器搜索到位置为 5 的 \ " ， 终点匹配器搜索到位置为 6 的 " ；
4. 子匹配器搜索到的位置更靠前，优先采用，并且下轮搜索从位置 7 开始；
5. 子匹配器搜索失败，终点匹配器搜索到位置为 10 的 " ；
6. 终点匹配器的结果被采用，结束匹配。

start-end 匹配器的强大之处在于起点匹配器、终点匹配器、子匹配器都是可以单独控制着色的，嵌套使用后可以处理非常复杂的情况。

```
{  
  start: {match: /<(\w+)/, 1: "error"}  
  end: {match: /(\w+)/>/, 1: "error"}  
  contains: [  
    {match: /\d+/, 0: "number"}  
    {match: /\w+/, 0: "meta"}  
  ]  
}
```

高亮结果如下：

- 123 abc <aaa abc 123 abc 123 bbb> abc 123

可以看到只有范围内的 abc 与 123 会被高亮。

快速入门

文件管理 ▾

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

到达文字末尾，才结束匹配。

如果将 `matchEndFirst` 属性设为 `true`，则算法是先匹配起点和终点，如果终点匹配失败则默认为匹配到文本结尾，然后起点和终点之间剩余的文本再使用子匹配器去匹配。

`endPriority` 属性

该属性仅在 `matchEndFirst` 为 `false` 时有效

上面提到终点匹配器与子匹配器进行竞争匹配，当终点匹配器与某个子匹配器匹配到相同位置时，默认会优先认为终点匹配器匹配成功，并结束匹配。

有些情况下，我们希望匹配到相同位置时，子匹配器优先级高于终点匹配器，那么就可以设置 `endPriority` 属性。

该属性值越大，则优先级越低，其代表的含义是优先级排在第 `N` 个子匹配器的后面，为 `0` 时则优先级排在所有子匹配器前面。

举个例子，在 C# 的 `@` 字符串中，字符 `"` 的转义是 `""`，例如：`@"abc""def"` 表示字符串 `abc"def`，如果使用以下匹配器则无法完全匹配：

```
{
  start: {match: /@"/}
  end: {match: /"/}
  style: "string"
  contains: [
    {match: /""/, 0: "strEscape"}
  ]
}
```

因为子匹配器的内容也会被终点匹配器匹配，导致子匹配器永远无法匹配到，因此我们需要把终点匹配器的优先级调低：

```
{
  start: {match: /@"/}
  end: {match: /"/}
  style: "string"
  endPriority: 1 // 优先级排在第 1 个子匹配器后面
  contains: [
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

EndMatcher 标记

如果你觉得 endPriority 不够直观，那么你可以在 contains 中添加 `<EndMatcher>` 标记，功能与 endPriority 一样：

```
{
  start: {match: /@"/}
  end: {match: /"/}
  style: "string"
  // endPriority: 1
  contains: [
    {match: /"/, 0: "strEscape"}
    <EndMatcher> // 优先级排在第 1 个子匹配器后面
  ]
}
```

`<EndMatcher>` 只能定义一次，且不能与 endPriority 属性同时定义！

Fail 标记

Fail 标记可以用于指定 contains 中的某个子匹配器匹配成功会使得整个 start-end 匹配器匹配失败。

例如我们想匹配一段以 `"""` 开头和结尾的文本，可以跨越多行，但中间不能存在空白行，如果要使用正则表达式实现该匹配，将会十分复杂。

```
// 匹配成功的例子
"""
abc
不存在空白行
"""

// 匹配失败的例子
"""
abc
```

快速入门

文件管理 ▾

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

而使用 start-end 匹配器配合 Fail 标记将很容易实现：

```
{
  start: {match: /"""/}
  end: {match: /"""/}
  style: "string"
  contains: [
    // 该正则表达式用于匹配空白行，Fail关键字不区分大小写
    {match: /( ?m)^\s*?$/} => FAIL
  ]
}
```

让我们理解一下这个过程：首先匹配开头 `""`，然后尝试去匹配结尾 `""` 和 空白行，如果在匹配到结尾 `""` 之前先匹配到 空白行，则整个匹配失败。

childrenSyntax 用法

新增于 2.26.9 版本，使用时建议在语法文件开头添加 `// require MT >= 2.26.9`

`childrenSyntax` 用于让 start-end 范围内的正文复用另一个已安装语法的规则。它与 `contains` 互斥：`contains` 使用当前语法定义的子匹配器处理正文，`childrenSyntax` 则把正文交给目标语法处理。

使用 `childrenSyntax` 时，外层语法必须先确定正文的终点边界，因此省略 `matchEndFirst` 会自动按 `true` 处理；如果显式设置为 `false`，语法文件将无法通过编译。

固定引用

目标既可以写语法名称，也可以写文件后缀名：

```
// require MT >= 2.26.9
{
  start: {match: /<script>/}
  end: {match: /<\s/script>/}
  childrenStyle: "default" // 可选
  childrenSyntax: ".js"
```

快速入门

文件管理 ▾

ROOT 相关

MT2 文件夹

书签手势说明

文件打开方式

压缩文件相关

批量重命名

文本编辑器

正则表达式

语法文件开发

Hex 编辑器

文件对比器

终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

上例也可以写成 `childrenSyntax: "JavaScript"` 。

动态引用

如果需要根据起点匹配器的捕获值动态选择语法，可以使用以下形式：

```
{
  start: {match: /<code\s+lang="([^"]+)">/}
  end: {match: /<\code>/}
  childrenStyle: "default" // 可选
  childrenSyntax: {
    capture: 1
    ignoreCase: true // 可选
    "js" => ".js"
    "css" => "CSS"
    "json" => ".json"
    "default" => ".json"
    default => ".js" // 可选
  }
  missingStyle: "comment" // 可选
}
```

其中 `capture` 表示读取起点 `match` 匹配器的哪个捕获组，映射右侧同样可以使用语法名称或后缀名。动态引用还需遵守以下规则：

- `start` 必须是直接的 `match` 匹配器，`capture` 必须指向有效的捕获组
- 动态 `childrenSyntax` 可以单独设置 `ignoreCase`，用于控制捕获值与映射键的比较是否区分大小写；映射键的重复检查也遵循相同规则。省略时继承语法文件的全局 `ignoreCase`
- `ignoreCase` 必须写在 `capture` 之后、所有 `include` 和映射项之前
- 映射键可以是空字符串，例如 `"" => ".js"`
- 裸 `default` 是可选的默认分支，最多只能定义一次；捕获组未参与匹配或捕获值没有命中显式分支时使用它
- 所有显式分支始终优先于裸 `default`，与书写位置无关；`"default"` 只是普通字符串分支，可以和裸 `default` 同时存在

快速入门

文件管理 ▾

ROOT 相关

MT2 文件夹

书签手势说明

文件打开方式

压缩文件相关

批量重命名

文本编辑器

正则表达式

语法文件开发

Hex 编辑器

文件对比器

终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

复用动态映射

同一组动态映射需要在多个 `childrenSyntax` 中使用时，可以先在 `defines` 中定义，再通过 `include` 引用：

```
defines: [  
  "commonLanguages": {  
    "js" => ".js"  
    "json" => ".json"  
  }  
  "extraLanguages": {  
    "js" => "JavaScript"  
    "css" => ".css"  
  }  
]  
contains: [  
  {  
    start: {match: /<code\s+lang="([^"]+)">/}  
    end: {match: /<\/code>/}  
    childrenSyntax: {  
      capture: 1  
      ignoreCase: true // 可选  
      include: "commonLanguages"  
      include: "extraLanguages"  
      "js" => ".js" // 可选的局部映射  
      default => ".json" // 可选  
    }  
  }  
]
```

复用映射遵循以下规则：

- 一个动态 `childrenSyntax` 可以引用多个映射，也可以只有 `include` 而没有局部映射和裸 `default`
- `include` 必须写在 `capture`、可选的 `ignoreCase` 之后，并全部位于局部映射和裸 `default` 之前；同一个动态块不能重复引用同名映射
- 合并优先级依次为：前一个 `include` < 后一个 `include` < 局部映射，因此后引用的映射可以覆盖先引用映射的同名分支
- 每个复用映射与局部映射分别算一层。同一层出现重复键会编译失败，跨层同名键则按上述优先级合法覆盖；大小写是否相同由使用处的 `ignoreCase` 决定

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

- `include` 指向不存在的定义，或者指向正则表达式、匹配器定义，都会导致语法文件编译失败

映射定义本身的写法和限制见[用于 childrenSyntax 映射](#)。

missingStyle 属性

`missingStyle` 只能与 `childrenSyntax` 配合使用。目标语法不存在不会阻止当前语法文件安装，以下情况会让正文使用 `missingStyle` 指定的风格：

- 显式分支匹配成功，但对应的目标语法不存在
- 选择了裸 `default`，但它引用的目标语法不存在
- 捕获值没有命中任何显式分支，并且没有定义裸 `default`

显式分支一旦匹配成功，即使它引用的目标语法不存在，也不会继续选择裸 `default`。如果省略 `missingStyle`，将使用 `childrenStyle`；如果 `childrenStyle` 也未定义，则使用 `style` 对应的正文风格。

嵌入语法除了负责正文高亮，还会为该范围提供注释切换、括号高亮和行背景规则；代码格式化仍使用外层语法的 formatter。

3. group 匹配器

```
{
  group: link | linkAll | select
  style: "风格名称，可空"
  contains: [
    子匹配器1
    子匹配器2
    .....
  ]
}
```

group 匹配器支持三种类型的匹配规则：`link`、`linkAll`、`select`

link 与 linkAll 规则

link 与 linkAll 规则要求子匹配组在匹配时必须首尾相连，之间不能有任何字符。

快速入门

文件管理 ▾

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

匹配成功。

举个简单例子：

```
{
  group: linkAll
  contains: [
    {match: /a+/}
    {match: /b+/}
    {match: /c+/}
  ]
}
```

这个匹配器可以匹配 `aabbcc` 但无法匹配 `aabb cc`，因为 `b` 和 `c` 没有头尾相连，中间还有个空格。

如果将 `linkAll` 改为 `link`，那么可以匹配 `aabbcc`、`aabb`、`aa`，因为只需要第一个子匹配器匹配成功就行，后续的子匹配器如果匹配失败将直接忽略。

看到这里你可能会觉得使用正则表达式 `/a+b+c+/` 就可以实现上面的匹配规则，不用这么麻烦，但是这里只是举了个简单的例子，实际运用时可以是其它类型的子匹配器，这是正则表达式无法实现的。

select 规则

`select` 规则是依次使用各个子匹配器去匹配，谁的匹配位置最靠前（匹配位置一样则定义在前面的子匹配器优先），就使用该子匹配器的匹配结果。

举个例子，你要高亮字符串、数字、变量，但有个前提：它们被括号包围起来，且之间是头尾相连的，那么可以配合 `linkAll` 规则这么写：

```
// 这里省略了 string、number、variable 的定义
{
  group: linkAll
  contains: [
    {match: /\(/}
    {
      group: select
```

快速入门

文件管理 ▾

[ROOT 相关](#)[MT2 文件夹](#)[书签手势说明](#)[文件打开方式](#)[压缩文件相关](#)[批量重命名](#)[文本编辑器](#)[正则表达式](#)[语法文件开发](#)[Hex 编辑器](#)[文件对比器](#)[终端模拟器](#)

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

```
        {include: "number"}
        {include: "variable"}
    ]
}
{match: /\)/}
]
}
```

不要单独使用 select 规则的 group 匹配器，这没有意义，因为写在 contains 内的匹配器也是类似的处理逻辑，正确的用法是配合其它匹配器去使用，就像上面的例子。

下面两个例子的实际效果完全一样，用 select 反而多此一举：

```
contains: [
  {
    group: select
    contains: [
      {include: "string"}
      {include: "number"}
      {include: "variable"}
    ]
  }
]
```

```
contains: [
  {include: "string"}
  {include: "number"}
  {include: "variable"}
]
```

4. number 匹配器

```
{
  number: "2|8|10|16|F|E|P|_|'"
  iSuffixes: "l" // 整数后缀，可选，默认为空
  fSuffixes: "f|d" // 浮点数后缀，可选，默认为空
  style: "风格名称" // 可选，默认为number
}
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

其中 **number** 用于指定匹配的数字格式，每个配置项之间使用 `|` 连接：

配置项	说明
2	匹配二进制数，即 <code>0b</code> 开头的数字，如： <code>0b101010</code>
8	匹配八进制数，即 <code>0o</code> 开头的数字，如： <code>0o777</code>
10	匹配十进制数，包括 <code>0</code> 开头的数字，如 <code>132</code> 、 <code>0777</code>
16	匹配十六进制数，即 <code>0x</code> 开头的数字，如： <code>0xFFFF</code>
F	匹配浮点数，如： <code>0.123</code>
E	匹配科学计数法数字，如： <code>1.321E10</code>
P	匹配十六进制浮点数，如： <code>0xFF.AAP123</code>
_	数字之间允许使用下划线连接，如 <code>1_2_3</code>
'	数字之间允许使用单引号连接，如 <code>1'2'3</code>

iSuffixes 和 **fSuffixes** 分别用于指定整数后缀与浮点数后缀，不区分大小写，每个后缀用 `|` 隔开。

- 二进制数、八进制数、十进制数、十六进制数会尝试匹配整数后缀
- 十进制数、浮点数、科学计数法数字、十六进制浮点数会尝试匹配浮点数后缀

后缀只会匹配一次，且不互相叠加，如果需要叠加则要手动列出所有情况。

例如 C 语言中，整数后缀 **L** 表示长整数，**U** 表示无符号整数，可叠加使用，为此你需要这么写：

```
{
  iSuffixes: "l|u|lu|ul"
}
```

快速入门

文件管理 ▾

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

关于八进制

许多编程语言使用 `0` 开头来表示八进制，而这里却使用 `0o` 开头来表示，其实 `0` 开头已经包含在十进制中，也就是说开启了十进制后，`0777` 就会被高亮。

当然这种情况下 `0888` 也会被高亮，这明显是一个错误的八进制数字，但在很多 IDE 中，`0888` 一样会被当作数字来高亮，只是它会提示你这是一个错误的数字。

基于这个理解，高亮 `0888` 并没有错，只是 MT 不是 IDE，不会提示你它存在语法错误。

如果你就是希望不高亮错误的八进制数字，那么可以自己编写正则表达式，这并不难。

```
// 高亮十进制与0开头的八进制，跳过错误的八进制
{match: /\b(0|0[0-7]+|[1-9][0-9]*)\b/, 0: "number"}
```

其它说明

很多编程语言的数字语法之间存在着些许差异，本匹配器主要是基于 Java 的语法标准，并没有针对其它语言提供细节选项，这会导致一些错误。

例如 C 语言中十进制整数不允许使用浮点数后缀，即 `123f` 是非法的；Python 中不允许使用连续的下划线，即 `1__2` 是非法的，而它们在 Java 中是合法的。

其实这和上面提到的八进制一样，我们的重点是高亮，这类非法的数字也完全可以被高亮，缺的只是一个语法错误提示，而提示语法错误更多是 IDE 的职责，所以如果你遇到这个问题，不必太过于纠结。

5. builtin 匹配器

```
{builtin: #内置的匹配器#}
```

builtin 匹配器可以直接调用 MT 内置的匹配器，包含一些常用的规则，如匹配字符串、匹配数字，拿来即用十分方便。

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

6. include 匹配器

```
{include: "自定义匹配器"}
```

引用在 defines 中定义的匹配器，具体请看[后面的介绍](#)。

★ 核心逻辑

语法引擎采用匹配器竞争机制来处理文本高亮，理解这个机制对于编写正确的语法文件非常重要。

匹配流程

- 引擎从文本位置 0 开始，依次使用 contains 中的各个匹配器进行匹配
- 找到匹配位置最靠前的匹配器，采用该匹配器的着色方案
- 跳转到匹配文本的末尾位置，继续下一轮匹配
- 重复以上步骤，直到处理完所有文本

优先级规则

当多个匹配器在同一位置都能匹配成功时，写在前面的匹配器优先级更高。

示例分析

假设我们需要高亮两种字符串：使用 `"""` 包裹的三引号字符串和使用 `""` 包裹的双引号字符串，有以下两个匹配器：

```
contains: [  
  {match: /"".*?"/, 0: "string"} // 匹配器A：三引号  
  {match: /"".*?"/, 0: "meta"}   // 匹配器B：双引号  
]
```

现在用这两个匹配器去匹配文本 `"""abc"""`，让我们分析匹配过程：

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

- 引擎从位置 0 开始，依次调用匹配器进行匹配
- 匹配器A 在位置 0 匹配成功，匹配到 `""abc""`（位置 0-8）
- 匹配器B 在位置 0 匹配成功，匹配到 `""abc"`（位置 0-7，注意 `.*` 是非贪婪匹配，匹配到第一个 `"` 就停止）
- 两个匹配器都在位置 0 匹配成功，由于匹配器A定义在前面，优先级更高，采用匹配器A的结果
- 跳转到位置 9，继续下一轮匹配

- 高亮结果：`""abc""`

如果我们把两个匹配器的顺序调换：

```
contains: [  
    {match: /"".*?""/, 0: "meta"}           // 匹配器B：双引号  
    {match: /"""".*?"""/, 0: "string"}       // 匹配器A：三引号  
]
```

再来分析匹配过程：

- 引擎从位置 0 开始，依次调用匹配器进行匹配
- 匹配器B 在位置 0 匹配成功，匹配到 `""abc""`（位置 0-7）
- 匹配器A 在位置 0 匹配成功，匹配到 `""""abc"""`（位置 0-8）
- 两个匹配器都在位置 0 匹配成功，由于匹配器B定义在前面，优先级更高，采用匹配器B的结果
- 跳转到位置 8，继续下一轮匹配
- 位置 8 只剩下一个 `"`，两个匹配器都无法匹配

- 高亮结果：`""abc""`

可以看到，由于匹配器顺序的问题，三引号字符串被错误地识别，最后一个 `"` 没有被高亮。

总结

- 在编写语法文件时，应该把更长或更具体的匹配规则放在前面
- 把更短或更通用的匹配规则放在后面
- 这样可以确保特殊情况优先被处理，避免被通用规则"抢走"

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

★ 文件结构一览

```
// require MT >= 2.26.9
{
  name: ["语法名称", ".后缀名1", ".后缀名2"]
  hide: false
  ignoreCase: false
  styles: [
    "风格名称", #RRGGBB, #RRGGBB
    "风格名称" > "已有的风格名称"
  ]
  comment: {startsWith: "/*", endsWith: "*/"}
  bracketPairs: [{"{", "[", "(")]
  lineBackground: {match: /正则表达式/, style: "风格名称"}
  defines: [
    // 这里可以定义正则表达式、匹配器或者 childrenSyntax
    "regex": /正则表达式/
    "number": {match: /\d+/, 0: "风格名称"}
    "string": [
      {match: /".*?"/, 0: "风格名称"}
      {match: /'.*?'/, 0: "风格名称"}
    ]
    "childrenSyntaxMap": {
      "js" => ".js"
      "css" => "CSS"
    }
  ]
  contains: [
    //
    // 此处内容是整个语法文件的核心，语法引擎会使用这里的匹
    //

    // match 匹配器
    {match: /正则表达式/, 0: "风格名称"}

    // start-end 匹配器：使用 contains 处理正文
    {
      start: {match: /正则表达式/}
      end: {match: /正则表达式/}
      style: "风格名称"
      childrenStyle: "风格名称"
      contains: [
        .....
      ]
    }
  ]
}
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

```
start: {match: /(正则表达式)/}
end: {match: /正则表达式/}
// 可选，省略时使用默认风格
style: "风格名称"
// 可选，省略时正文沿用 style; style 未定义时使用
childrenStyle: "风格名称"
// 也可以直接写 childrenSyntax: "语法名称或后缀"
childrenSyntax: {
  capture: 1
  include: "childrenSyntaxMap"
}
// 可选，省略时沿用 childrenStyle; childrenStyle 未定义时使用
missingStyle: "风格名称"
}

// number 匹配器
{
  number: "2|8|10|16|F|E|P|_|'"
  iSuffixes: "整数后缀"
  fSuffixes: "浮点数后缀"
  style: "风格名称"
}

// builtin 匹配器
{builtin: #内置的匹配器#}

// 引用 defines 中的正则表达式
{match: include("regex"), 0: "风格名称"}

// 引用 defines 中的匹配器
{include: "number"}
]
codeFormatter: #内置的代码格式化器#
codeShrinker: #内置的代码压缩器#
}
```

属性：name

```
name: ["语法名称", ".后缀名1", ".后缀名2"...]
```

指定语法名称与文件后缀，第一个字符串为语法名称，第二个开始均为后缀名，后缀名可以有多个。

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

属性：hide

```
hide: true | false
```

设置是否隐藏，隐藏后在文本编辑器中手动选择语法时将无法看到该语法，默认为 false。

属性：ignoreCase

```
ignoreCase: true | false
```

设置是否不区分大小写，设置为 true 后，所有正则表达式在匹配时均不会区分大小写，默认为 false。

如果只是想要部分正则表达式不区分大小写，可使用 i 标志，如 `/(?i)[a-z]+/`

属性：styles

```
styles: [  
    // 定义前景色，第一个是日间模式颜色，第二个为夜间模式颜色  
    "风格名称", #RRGGBB, #RRGGBB  
  
    // 若两个颜色数值连接在一起则分别表示前景色和背景色  
    "风格名称", #RRGGBB#RRGGBB, #RRGGBB#RRGGBB  
  
    // 把背景色前面的 # 换成 $ 则表示行背景色（具体看下文）  
    "风格名称", #RRGGBB$RRGGBB, #RRGGBB$RRGGBB  
  
    // 可以添加@标记来设置 B:粗体 I:斜体 U:下划线 S:删除线  
    "风格名称", #RRGGBB, #RRGGBB, @BIUS  
  
    // 可以不指定颜色，单独设置@标记  
    "风格名称", @U  
  
    // 可以继承已有的风格  
    "风格名称" > "已有的风格名称"  
  
    // 继承之后可以添加@标记
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

MT 已经内置了一些常用风格，具体请看[附录](#)。

颜色格式

支持 #RGB、#RRGGBB、#ARGB、#AARRGGBB 四种颜色格式，单写一个颜色表示前景色，如果连在一起写则表示前景色和背景色，例如：`#FFFFFF#000000`

如果你只想单独定义背景色，则需要写成 `#NULL#背景色` 来表示前颜色未定义。

行背景色

将背景色前面的 `#` 换成 `$` 将变为表示行背景色。

不能同时指定背景色和行背景色，并且行背景色不支持透明度

某个字符一旦拥有行背景色，那么在渲染时，该颜色将作为其所在行的背景色。

以下是为 `.method` 添加了行背景色 `#800000$FFEDED` 的效果：

```
5 # virtual methods
6 .method public abstract shouldSkipClass(Ljava/lang/Class;Ljava/lang/Class;)Z
7     .annotation system Ldalvik/annotation/Signature;
8     value = {
9         "(",
10         "Ljava/lang/Class<",
11         "**>;)Z"
12     }
13     .end annotation
14 .end method
15
16 .method public abstract shouldSkipField(Lcom/google/common/collect/Map;)Z
17 .end method
18
```

如果同一行中有多个字符拥有行背景色，将以第一个字符为准。

该功能是 [属性：lineBackground](#) 的增强版，可以更加灵活地指定行背景。

风格的向下传播

所有匹配器定义的风格会向下传播给它的子匹配器，具体请参考以下例子：

快速入门

文件管理 ▾

ROOT 相关

MT2 文件夹

书签手势说明

文件打开方式

压缩文件相关

批量重命名

文本编辑器

正则表达式

语法文件开发

Hex 编辑器

文件对比器

终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

```
end: {match: /\\"/}
style: "string"
contains: [
  {match: /\\"./, 0: "strEscape"}
]
```

其中父匹配器的风格是 `string`，子匹配器的风格是 `strEscape`

1. 假如 `string` 定义了背景色，而 `strEscape` 没有定义背景色，最终效果是子匹配器会使用 `string` 的背景色，对于前景色也是如此。
2. 假如 `string` 定义了粗体、斜体、下划线、删除线，那么即使 `strEscape` 没有这些定义，子匹配器的文字也会被附上粗体、斜体、下划线、删除线。
3. 要在某个点切断风格传播，只需在使用某个风格时在其前面加上 `!!`，就像这样：

```
{
  start: {match: /\\"/}
  end: {match: /\\"/}
  style: "string"
  contains: [
    {match: /\\"./, 0: "!!strEscape"}
  ]
}
```

这样 `strEscape` 就不会受到 `string` 以及更上级风格的影响。

属性：comment

```
// 行注释
comment: {startsWith: "//"}
// 块注释
comment: {startsWith: "/*", endsWith: "*/"}
```

定义注释，支持行注释与块注释。

定义注释之后，语法引擎会自动添加正则表达式对注释内容进行高亮，您无需额外配置，并且可同时开启该语法的注释切换功能，当

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

默认在切换为注释时，在注释符号代码的后面会自动添加一个空格，如果不希望增加空格，可配置 `insertSpace` 选项。

```
comment: {startsWith: "//", insertSpace: false}
```

原内容

abc

切换注释 (insertSpace: true)

// abc

切换注释 (insertSpace: false)

//abc

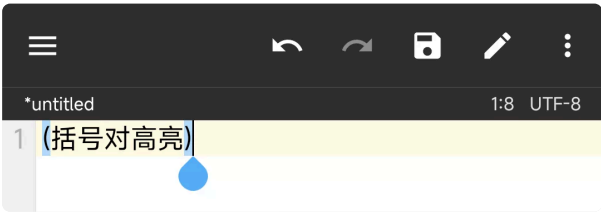
另外如果你仅仅只想开启注释切换功能，不希望语法引擎自动添加正则表达式对注释内容进行高亮，可添加 `addToContains` 选项。

```
comment: {startsWith: "//", insertSpace: false, addToContains: true}
```

注意，`startsWith`、`endsWith`、`insertSpace`、`addToContains` 这四个选项的先后顺序不能变，也就是说 `addToContains` 不能放在 `insertSpace` 的前面。

属性：bracketPairs

在文本编辑器中，当你把光标放在一个括号上时，与其对应的另一个括号将会自动高亮：



默认情况下会自动高亮括号对 `{ }` `[ ]` `( )`，你可以通过 `bracketPairs` 属性来指定要高亮的括号对：

```
bracketPairs: ["{}", "[]", "()"]
```

配置中所填写的括号对文本必须是两个字符，且两个字符不能一样！

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式

语法文件开发

- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

属性：lineBackground

lineBackground: {match: /正则表达式/, style: "风格名称, 使

如果正则表达式可以完全匹配某一行的文本内容，则为该行添加背景颜色。

例如Smali语法中会对 .method 所在行进行高亮：

// 注意：这里的 methodHeader 不是内置颜色，需单独定义
lineBackground: {match: /\[\t \]*\.method .*/, style: "me

```
5 # virtual methods
6 .method public abstract shouldSkipClass(Ljava/lang/Class;
7     .annotation system Ldalvik/annotation/Signature;
8     value = {
9         "(",
10        "Ljava/lang/Class<",
11        "**>;)Z"
12    }
13 .end annotation
14 .end method
15
16 .method public abstract shouldSkipField(Lcom/google/android/apktool/library/dex/ClassDef;
17 .end method
18
```

属性：contains

```
contains: [
    // match 匹配器
    {match: /正则表达式/, 0: "风格名称"}

    // start-end 匹配器
    {
        start: {match: /正则表达式/}
        end: {match: /正则表达式/}
        style: "风格名称"
        contains: [
            .....
        ]
    }

    // number 匹配器
    {
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

```
    suffixes: 为无数后缀
    style: "风格名称"
  }

  // builtin 匹配器
  {builtin: #内置的匹配器#}

  // 引用 defines 中的正则表达式
  {match: include("regex"), 0: "风格名称"}

  // 引用 defines 中的匹配器
  {include: "number"}
]
```

你需要在 contains 内编写你的匹配器，它是整个语法文件的核心，语法引擎会使用这里的匹配器去查找并高亮文本。

关于匹配器已在上文有详细的介绍，这边不再重复。

属性：defines

你可以在 defines: [] 块中定义正则表达式、匹配器或者 childrenSyntax 动态映射，然后在其它地方引用。三种定义共用同一个命名空间，定义名不能重复。

用于正则表达式

举个例子，你需要把使用 () 包裹起来的邮箱地址高亮为红色，使用 [] 包裹起来的邮箱地址高亮为绿色，那么可以这么写：

```
contains: [
  {match: /\(^[\w\.-]+@[\w\.-]+\.\w+$\)\/, 0: "error"},
  {match: /\^[^[\w\.-]+@[\w\.-]+\.\w+$\]\/, 0: "string"}
]
```

这样写没有问题，但不利于维护，如果你后面发现匹配邮箱地址的正则表达式无法匹配一些特殊邮箱，需要进行完善，就得修改两处代码。

而实际情况可能比这更复杂，可能要修改不止两处代码，可能要在一段复杂的正则表达式中找出需要修改的片段将它替换掉，这样非

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

因此你可以把整个正则表达式直接复制出来放到 defines 中。

```
defines: [
  "emails": /[\\w\\.-]+@[\\w\\.-]+\\.\\w+/
]
contains: [
  {match: /\(/ + include("email") + /\)/, 0: "error"}
  {match: /\[/ + include("email") + /\]/, 0: "string"}
]
```

这样后期维护就比较简单不容易出错了。

defines 中定义的正则表达式也可以引用在它前面定义的正则表达式：

```
defines: [
  "exp1": /正则表达式1/
  "exp2": include("exp1") + /正则表达式2/
]
```

被引用的表达式必须在前面，像下面这样则不行：

```
defines: [
  "exp2": include("exp1") + /正则表达式2/
  "exp1": /正则表达式1/
]
```

用于匹配器

在 defines 中，你可以这样定义一个匹配器：

```
defines: [
  "number": {match: /\d+/, 0: "风格名称"}
]
```

或者这样定义一个匹配器组：

```
defines: [
  "string": [
    {match: /\.*/, 0: "风格名称"}
  ]
]
```

快速入门

文件管理 ▾

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

然后在 contains 中引用：

```
contains: [  
  {include: "number"}  
  {  
    // 所有使用匹配器的地方都可以使用 include 代替  
    start: {include: "string"}  
    end: {include: "string"}  
  }  
]
```

支持自递归引用：

```
defines: [  
  "group": {  
    start: {match: /</, 0: "风格名称"}  
    end: {match: />/, 0: "风格名称"}  
    contains: [  
      {include: "group"}  
    ]  
  }  
]
```

也可以交叉递归引用：

```
defines: [  
  "group1": {  
    start: {match: /</, 0: "红色"}  
    end: {match: />/, 0: "红色"}  
    contains: [  
      {include: "group2"}  
    ]  
  }  
  "group2": {  
    start: {match: /</, 0: "绿色"}  
    end: {match: />/, 0: "绿色"}  
    contains: [  
      {include: "group3"}  
    ]  
  }  
  "group3": {
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

```
contains: [
  {include: "group1"}
]
}
```

你可以下载安装[这个例子](#) 来体会交叉递归引用的妙用。

与定义正则表达式不同，定义匹配器时可以随意交叉引用，不需要考虑先后顺序。

但是在交叉引用时，注意不要出现循环硬性依赖，通俗的讲就是**匹配器1**查找成功的前提是**匹配器2**查找成功，而**匹配器2**查找成功的前提又是**匹配器1**查找成功，这样就出现死循环。

例如下面这个例子：

```
defines: [
  "xxx": {
    start: {include: "yyy"}
    end: {match: /.*/}
  }
  "yyy": {
    start: {include: "xxx"}
    end: {match: /.*/}
  }
]
contains: [
  {include: "xxx"}
]
```

在安装时将会收到一个错误：

```
There is a circular hard dependency: include "xxx" >
include "yyy" > include "xxx"
```

用于 childrenSyntax 映射

在 `defines` 中还可以定义一组可复用的 `childrenSyntax` 显式映射：

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

```
    "" => ".js"
    "js" => "JavaScript"
    "css" => ".css"
    "default" => ".json"
  }
]
contains: [
  {
    start: {match: /<code\s+lang="([^"]*)">/}
    end: {match: /<\/code>/}
    childrenSyntax: {
      capture: 1
      include: "webLanguages"
    }
  }
]
```

映射定义至少需要一个显式的 "选择值" => "目标语法" 分支。选择值允许为空字符串，目标语法不能为空；目标语法既可以使用名称，也可以使用后缀名。

映射定义中不能出现 capture 、 ignoreCase 、裸 default 或嵌套的 include 。其中 "default" 是合法的普通字符串选择值，只有不带引号的裸 default 不能写在映射定义中。

映射定义本身不携带大小写规则；被 childrenSyntax 引用后，所有映射键统一按该使用处的 ignoreCase 比较和判重。完全相同的选择值始终不能在同一映射中重复；仅大小写不同的选择值能否同时存在，则取决于使用处是否区分大小写。使用处没有单独声明 ignoreCase 时，继承整个语法文件的全局设置。

映射的组合与覆盖规则见[复用动态映射](#)。

这种定义只能由动态 childrenSyntax 中的 include: "映射名称" 引用，不能作为 {include: "匹配器名称"} 的匹配器，也不能由正则表达式中的 include("正则名称") 引用。

属性：codeFormatter

codeFormatter: #内置的代码格式化器#

快速入门

文件管理 ▾

ROOT 相关

MT2 文件夹

书签手势说明

文件打开方式

压缩文件相关

批量重命名

文本编辑器

正则表达式

语法文件开发

Hex 编辑器

文件对比器

终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶

目前仅支持指定内置的代码格式化器，**不支持自定义**，具体包含 CSS、HTML、Java、JavaScript、JSON、XML、Smali 代码格式化器，分别对应：

```
#BUILT_IN_CSS_FORMATTER#  
#BUILT_IN_HTML_FORMATTER#  
#BUILT_IN_JAVA_FORMATTER#  
#BUILT_IN_JS_FORMATTER#  
#BUILT_IN_JSON_FORMATTER#  
#BUILT_IN_XML_FORMATTER#  
#BUILT_IN_SMALI_FORMATTER#
```

属性：codeShrinker

```
codeShrinker: #内置的代码压缩器#
```

指定该语法的代码压缩器，设置后可以在文本编辑器中使用代码压缩功能。

目前仅支持指定内置的代码压缩器，**不支持自定义**，具体包含 CSS、HTML、JSON 代码格式化器，分别对应：

```
#BUILT_IN_CSS_SHRINKER#  
#BUILT_IN_HTML_SHRINKER#  
#BUILT_IN_JSON_SHRINKER#
```

指定最低 MT 版本

随着语法规则的不断更新，新编写的语法文件可能无法在旧版 MT 上安装，而安装时的错误信息也会让用户摸不着头脑，不知道是由于 MT 版本过低导致的。

为了优化这个问题，你可以在语法文件中指定最低 MT 版本，只需在文件开头添加一行注释即可：

```
// require MT >= 2.16.0  
{
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

其中的 `>=` 也可以换成 `>`，前者是大于等于判断，后者是大于判断。

其中的 `2.16.0` 是版本名称，你也可以换成八位的版本号如 `24052969` 来指定，不过更推荐使用版本名称，会更加直观。

安装时如果当前 MT 版本低于语法文件要求的版本：

- 当前 MT 版本 `< 2.16.5`：如果存在不兼容语法会出现编译错误信息
- 当前 MT 版本 `>= 2.16.5`：提示用户更新 MT 版本再安装该语法文件

虽然 MT 版本 `< 2.16.5` 时不会提示用户去更新 MT 版本，但只要用户略懂英语并查看了语法文件内容，大概率会知道是 MT 版本过低的原因。

预处理指令

从 2.19.4 版本开始，语法文件支持预处理指令，可以根据 MT 版本进行条件编译。这使得你可以在一个语法文件中针对不同版本的 MT 提供不同的实现。

支持的指令

预处理指令必须单独占一行，且以 `#` 开头（前面可以有空白字符）。支持以下指令：

- `#if` - 条件判断开始
- `#elif` - 否则如果（可选，可多个）
- `#else` - 否则（可选，只能有一个）
- `#endif` - 条件判断结束
- `#error` - 触发编译错误

条件表达式

条件表达式的格式为：`MT 运算符 版本`

支持的运算符：

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

- < - 小于
- <= - 小于等于
- == - 等于
- != - 不等于

版本可以是：

- 版本名称格式： 2.19.4
- 版本号格式： 25120278 （八位数字）

用法示例

基本用法

注意这里只是演示，实际上下面的 if 条件永远成立，因为预处理指令从 2.19.4 版本才开始支持。

```
// 根据MT版本使用不同的匹配器
{
    name: ["示例", ".example"]
    contains: [
        #if MT >= 2.19.0
        // MT 2.19.0及以上版本使用的匹配器
        {
            number: "2|8|10|16|F|E|P|_|'"
            style: "number"
        }
        #else
        // 旧版本使用的匹配器
        {match: /\d+/, 0: "number"}
        #endif
    ]
}
```

多条件判断

```
{
    name: ["示例", ".example"]
    contains: [
        #if MT >= 2.19.0
        // 2.19.0及以上版本的代码
        {builtin: #BUILT_IN_STRING#}
        #elif MT >= 2.18.0
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

```
#endif

// 2.18.0以下版本的代码
{match: /^[^"]*"/, 0: "string"}
#endif

]

}
```

error 指令

可以使用 `#error` 指令在特定条件下阻止语法文件安装：

```
{
    name: ["示例", ".example"]

    #if MT < 2.19.0
    #error 此语法文件需要 MT 2.19.0 或更高版本
    #endif

    contains: [
        // ...
    ]
}
```

嵌套使用

预处理指令可以嵌套使用：

```
{
    name: ["示例", ".example"]
    contains: [
        #if MT >= 2.19.0
            #if MT >= 2.19.4
                // MT 2.19.4及以上
                {number: "2|8|10|16|F|E|P|_|'", style: "number"}
            #else
                // MT 2.19.0到2.19.4之间
                {number: "2|8|10|16|F|E|P|_", style: "number"}
            #endif
        #else
            // MT 2.19.0以下
            {match: /\d+/, 0: "number"}
        #endif
    ]
}
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

注意事项

- 1. 预处理指令必须单独占一行
- 2. #else 和 #endif 后面不能有其他内容（空白除外）
- 3. 每个 #if 必须有对应的 #endif
- 4. #elif 和 #else 必须在 #if 和 #endif 之间
- 5. 同一个 #if 块中只能有一个 #else

与 require 指令的区别

- require 指令用于指定语法文件的最低版本要求，如果不满足会阻止安装
- 预处理指令用于在同一个文件中为不同版本提供不同实现，可通过 #error 阻止安装
- 预处理指令也可以实现 require 指令的功能，但推荐优先使用 require 指令，因为它写起来更简洁，错误提示对用户也更加友好

安装语法高亮文件

将写好的语法规则保存为 .mtsx 文件，在 MT 管理器内打开它即可安装。

附录

内置的风格

内置的风格可在 MT.apk/assets/syntax/init/styles.mtsx 文件中找到，请以安装包内文件为准。

"default"	#000000	#A9B7C6
"string"	#067D17	#6A8759
"strEscape"	#0037A6	#CC7832
"comment"	#8C8C8C	#808080
"meta"	#9E880D	#BBB529
"number"	#1750EB	#6897BB
"keyword"	#0033B3	#CC7832
"keyword2"	#800000	#AE8ABE
"constant"	#871094	#9876AA

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

"operator"	#205060	#508090
"propKey"	#083080	#CC7832
"propVal"	#067D17	#6A8759
"tagName"	#0030B3	#E8BF6A
"attrName"	#174AD4	#BABABA
"namespace"	#871094	#9876AA
"error"	#F50000	#BC3F3C

内置的匹配器

内置的匹配器及其具体规则可在 `MT.apk/assets/syntax/init/builtins.mtsx` 文件中找到，请以安装包内文件为准。

```
//
// 普通字符串
//

// 匹配转义符号，仅包含\x
#ESCAPED_CHAR#

// 单引号字符串
#SINGLE_QUOTED_STRING#

// 双引号字符串
#DOUBLE_QUOTED_STRING#

// 单双引号字符串
#QUOTED_STRING#

//
// Java字符串
//

// 匹配Java转义符号，包含 \b \t \n \f \r \" \' \\ \000 \u0000
#JAVA_ESCAPED_CHAR#

// Java单引号字符串
#JAVA_SINGLE_QUOTED_STRING#

// Java双引号字符串
#JAVA_DOUBLE_QUOTED_STRING#

// Java单双引号字符串
```

快速入门

文件管理

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改

逆向实战

插件开发

```
//  
// C字符串  
//  
  
// 匹配C转义符号, 包含 \b \t \n \f \r \x \" \' \\ \000 \u0000  
#C_ESCAPED_CHAR#  
  
// C单引号字符串  
#C_SINGLE_QUOTED_STRING#  
  
// C双引号字符串  
#C_DOUBLE_QUOTED_STRING#  
  
// C单双引号字符串  
#C_QUOTED_STRING#  
  
//  
// 数字相关  
//  
  
// 匹配数字, 支持整数与小数  
#NORMAL_NUMBER#  
  
// 编程语言数字, 大多数语言适用  
#PROGRAM_NUMBER#  
  
// 编程语言数字, 与上面比取消了二进制支持  
#PROGRAM_NUMBER2#  
  
// Java数字  
#JAVA_NUMBER#  
  
// C语言数字  
#C_NUMBER#
```

LSP 插件

支持为 VSCode 和 Vim 添加 mtsx 文件的语法高亮与代码补全功能（由 A4-Tacks 贡献）：

<https://github.com/A4-Tacks/mtsx-ls>



快速入门

文件管理 ▼

- ROOT 相关
- MT2 文件夹
- 书签手势说明
- 文件打开方式
- 压缩文件相关
- 批量重命名
- 文本编辑器
- 正则表达式
- 语法文件开发
- Hex 编辑器
- 文件对比器
- 终端模拟器

MCP 服务

逆向修改 ▶

逆向实战 ▶

插件开发 ▶